

Deterministic Indian Address Matching System

High-Level Project Overview

This document is a short, non-technical-friendly overview of the address matching system: what it does, how the code is organized, what extraction rules drive it, what the output fields mean, and which problems are known and intentionally left unsolved. It is not a full engineering spec — it is meant to be read in a few minutes.

1. What This System Does

Given two Indian addresses as free text, the system decides how confidently they refer to the same physical location — without using machine learning, embeddings, or any trained model. Every decision is made by explicit, readable rules. The system does two separate jobs:

- **Extraction** — pull out the flat/unit number, wing/block letter, and PIN code from the raw address text, leaving everything else (locality, street, landmark, city) as a leftover 'base address'.
- **Matching** — compare two addresses' extracted pieces and apply a business policy to produce a confidence tier (HIGH / MEDIUM / LOW) plus an explanation.

2. File-by-File Reference

Extraction & normalization

File	What it does
<code>unit_extractor.py</code>	Core, actively-developed extractor. Reads a raw address and pulls out UNIT, SUBUNIT, PIN, and BASE_ADDRESS using the rule set described in Section 3. This is the newest and most accurate extraction logic.
<code>parser.py</code>	Older, broader parser that recognizes ~12 different address roles (PIN, floor, flat, wing, building, street, landmark, city, etc). Kept unchanged; still used by parts of the pipeline that need the fuller breakdown.
<code>numeric.py</code>	Shared low-level helpers for detecting and canonicalizing numeric/PIN-like tokens (e.g. OCR-tolerant PIN recovery).
<code>canonical.py</code>	Normalizes numeric identifiers into a consistent comparable form (e.g. stripping leading zeros) without corrupting multi-part compounds.
<code>ocr_cleanup.py</code>	Light text cleanup to correct common OCR mistakes (e.g. letter/digit confusion) before extraction runs.

Matching, scoring & delivery

File	What it does
<code>datamodel.py</code>	Defines the shared data structures used everywhere: component types, confidence levels, evidence states, contradiction records, and the final match/policy result objects.
<code>comparator.py</code>	Compares one extracted component (e.g. two UNIT values) between address A and address B and produces an evidence verdict (match / mismatch / missing / etc).
<code>contradiction.py</code>	Looks at the comparator's evidence and flags meaningful contradictions (e.g. a confident flat number mismatch) versus harmless gaps.
<code>scorer.py</code>	Combines all the component-level evidence into an overall similarity signal for a pair of addresses.
<code>policy_engine.py / policies.py</code>	Business rules layer. Each named policy (Default, FlatOnly, WingOnly, StrictFlatAndWing) decides the final confidence tier from the evidence, preserving the exact behavior of the original legacy system except where an intentional, documented fix was made.

File	What it does
<code>legacy_mapping.py</code>	Translates new-style results back into the old system's output field names/labels, so existing consumers of the old API are unaffected.
<code>integration_adapter.py</code>	Thin glue layer that feeds <code>unit_extractor.py</code> 's UNIT/SUBUNIT/PIN output into the existing comparator/scorer/policy pipeline, without changing that pipeline's own logic.
<code>streamlit_app.py</code>	Interactive web UI: compare a single pair of addresses, or upload a CSV of address pairs for batch matching.
<code>address_old.py</code> , <code>matching_policies_old.py</code>	The original legacy implementation. Kept untouched, used only as a reference to verify the new system reproduces old behavior.

3. Extraction Rules (`unit_extractor.py`)

The extractor treats every number/letter token as a **candidate** first. A candidate only becomes UNIT or SUBUNIT when there is positive evidence for it — never just because of its shape or position. The main rule groups:

- **Keyword-anchored UNIT** — words like Flat, Flat No, Unit, Apartment, Shop, Gala, Room, House No, D.No, Property No directly in front of a value make it UNIT with high confidence (ANCHORED).
- **Keyword-anchored SUBUNIT** — Wing, Block, Tower directly claim a value as SUBUNIT (ANCHORED). A reverse form is also supported: the value can come *before* the keyword, e.g. 'C WING' or 'E BLOCK' (Tower is not supported in this reverse form).
- **Negative context** — words like Sector, Phase, Pocket, Floor, Road, Street, Lane, etc. block a nearby number from being read as UNIT, since it clearly means something else.
- **Compound splitting** — a single-letter prefix or suffix attached to a number (e.g. D-817, 701B, 18/C) is split into UNIT + SUBUNIT with lower confidence (INFERRED), even if a keyword claimed the whole compound (e.g. 'Flat 701B' still splits). Multi-letter or multi-part compounds (e.g. AB-101, 703-A1-66) are deliberately left whole — not split.
- **Leading-identifier inference** — when an address starts with a plausible flat/house number and no keyword is present (e.g. '801, SILVER SPRINGS...'), the number can still be inferred as UNIT (INFERRED), guarded against false positives like ordinals (5TH), abbreviations (P.O.), or a lone letter mistaken for an identifier.
- **PIN extraction** — a 6-digit PIN code is recognized independently (with OCR-tolerant recovery for near-miss values) and is never confused with UNIT.
- **BASE_ADDRESS** — built by removing only the confidently-extracted text spans from the *original* address string. It is never rebuilt from leftover fragments, so locality/street/landmark text stays intact and readable.

4. What the Output Fields Mean

Field / Status	Meaning
UNIT	The extracted flat/house/shop/unit number.
SUBUNIT	The extracted wing/block/tower identifier.
PIN	The extracted 6-digit postal code.
BASE_ADDRESS	Everything left in the address after UNIT/SUBUNIT/PIN text is removed (building name, locality, street, city, etc.).
ANCHORED	The value was directly claimed by an explicit keyword (e.g. 'Flat 401'). Highest confidence.
INFERRED	The value was deduced from structure/position without an explicit keyword (e.g. a leading number, or a split compound like '18/C'). Lower confidence than ANCHORED.

Field / Status	Meaning
UNKNOWN	No rule found enough evidence to assign a role; the token stays inside BASE_ADDRESS instead of being guessed.
AMBIGUOUS	Two or more candidates independently and equally claim the same role (e.g. 'Flat 101, Unit 202') and the system deliberately refuses to guess which one is correct.
MATCH / MISMATCH	Whether address A and address B agree or disagree on a given component.
MISSING_ON_A / B / BOTH	One or both addresses simply did not have that component extracted.
UNCERTAIN	Both sides only have INFERRED (guessed, not keyword-confirmed) values that disagree — treated more leniently than a confirmed ANCHORED mismatch, since neither side is fully sure to begin with.
Confidence Tier (HIGH / MEDIUM / LOW)	The final policy-level verdict for a given address pair, combining all component evidence.

5. Matching Policies

Four business policies are supported, each answering the question 'how strict should we be?' differently:

DefaultMatchingPolicy (general-purpose, balances flat/wing/PIN evidence), **FlatOnlyPolicy** (only cares about the flat number matching), **WingOnlyPolicy** (only cares about the wing/block matching), and **StrictFlatAndWingPolicy** (requires both flat and wing to agree for a high result). All four were verified to reproduce the original legacy system's behavior exactly, except for a small number of explicitly documented and intentional fixes.

6. Known, Unsolved Limitations

These are deliberately left as-is rather than force-fixed, because no safe general rule was found without risking new false positives elsewhere:

- **'C/O ... 30B SHED NO A/1...' style filler clauses** — the compound-split rule has no positional limit, so this produces an AMBIGUOUS result rather than a clean split.
- **'4B 308' interaction** — two extraction rules interact in a way that happens to produce a plausible-looking answer, but via the wrong mechanism internally.
- **'911 G T ROAD'** — a false positive where the 'G' in 'G T Road' (Grand Trunk Road) is misread as a wing letter. Two different narrow fixes were tried and both broke real, genuine cases elsewhere, so this is left unresolved.
- **'Flat No. 40 A'** — a bare letter after a keyword-claimed number is sometimes lost. Real examples in the data conflict with each other (one wants the letter kept as wing, another wants it dropped as part of a place name), so no confident fix exists yet.
- **Tower as reverse-order SUBUNIT** (e.g. '2 TOWER') — intentionally not supported yet; treated as an open business question, not a bug.
- **House / Plot as UNIT keywords** — deferred, since these words very often appear as part of a building/business name rather than an actual unit reference.
- **BASE_ADDRESS is not yet used in scoring** — today only UNIT/SUBUNIT/PIN affect the match confidence. Comparing the remaining address text (locality/street/landmark) is a planned future phase, not yet built.

All rules above are deterministic (plain conditional logic and pattern matching). No machine learning, embeddings, or trained models are used anywhere in this system.